

RL 기반 서버리스 오토스케일링

Azure Functions 2019 실제 워크로드 기반 Warm 인스턴스 수 최적화

GitHub: <https://github.com/1three/sogang-rl-autoscaling>

강화학습 프로젝트 | 서강대학교 인공지능학과

김한슬 (A74029)

| 목차

01 문제 정의 및 가설

02 데이터셋 및 환경 구성

03 MDP (Markov Decision Process) 설계

04 알고리즘 및 실험 설정

05 실험 결과 분석

06 알고리즘 분석 및 핵심 발견

07 결론 및 향후 과제

| 문제 정의 및 가설

서버리스 환경의 트레이드오프는 스케일링 딜레마다.

서버리스 환경에서 **Warm 인스턴스를 얼마나 유지할지**는 항상 트레이드오프를 동반합니다.

규칙 기반 오토스케일러(HPA)는 이 균형을 정적 임계값으로 다루기 때문에 수요가 불규칙할수록 낭비가 커집니다.

- **Warm 인스턴스 증가:** 콜드스타트 감소, 비용 증가
- **Warm 인스턴스 감소:** 비용 감소, 콜드스타트 증가

이 트레이드오프를 매 분 상황에 맞게 동적으로 최적화하기 위해 이 프로젝트를 시작하게 되었습니다.

- **H1: RL이 같은 비용에서 HPA보다 콜드스타트를 줄일 수 있을까?**
- **H2: RL이 하루 주기 트래픽 패턴을 학습해 선제적으로 스케일링할 수 있을까?**
- **H3: Reward 가중치로 비용-SLA 균형을 명시적으로 조절할 수 있을까?**

| 데이터셋 및 환경 구성

Azure Functions 2019 실제 트레이스를 환경으로 삼아,

매 분 warm 인스턴스 수를 결정하는 RL 에이전트를 학습시키고 기존 방식과 비교하는 방법으로 접근하였습니다.



Azure Data 2019

총 14일간의 분당 호출 수, 평균
실행시간, 메모리 사용량 데이터
활용 (Train 10일 / Test 4일)



동시성 계산 (Little's Law)

필요한 동시 인스턴스 수를
산출합니다.

$$C_t = \frac{\lambda_t \times d}{60000}$$



3가지 시나리오

Steady: 뚜렷한 하루 주기 반복

Spiky: 갑작스러운 호출 급증

Bursty: 상시 고부하 및 높은 변동성

MDP (Markov Decision Process) 설계

상태 (State) 및 행동 (Action)

State (16차원): 최근 10분 호출량,
현재 Warm 인스턴스 수, 직전 이용률,
최근 콜드스타트 비율 및 시간적 특성(sin/cos)으로 설정,
미래 정보는 배제하여 현실성을 확보했습니다.

Action (이산형 7개): 상대적 변화량으로 스케일링을
결정합니다.

$$a_t \in \{ -4, -2, -1, 0, +1, +2, +4 \}$$

보상 (Reward) 및 에피소드

Reward: 비용과 콜드스타트 패널티 간의 명시적 조절

$$r_t = - (w_{\text{cost}} \cdot n_{t+1} + w_{\text{cold}} \cdot \text{coldstart}_{t+1})$$

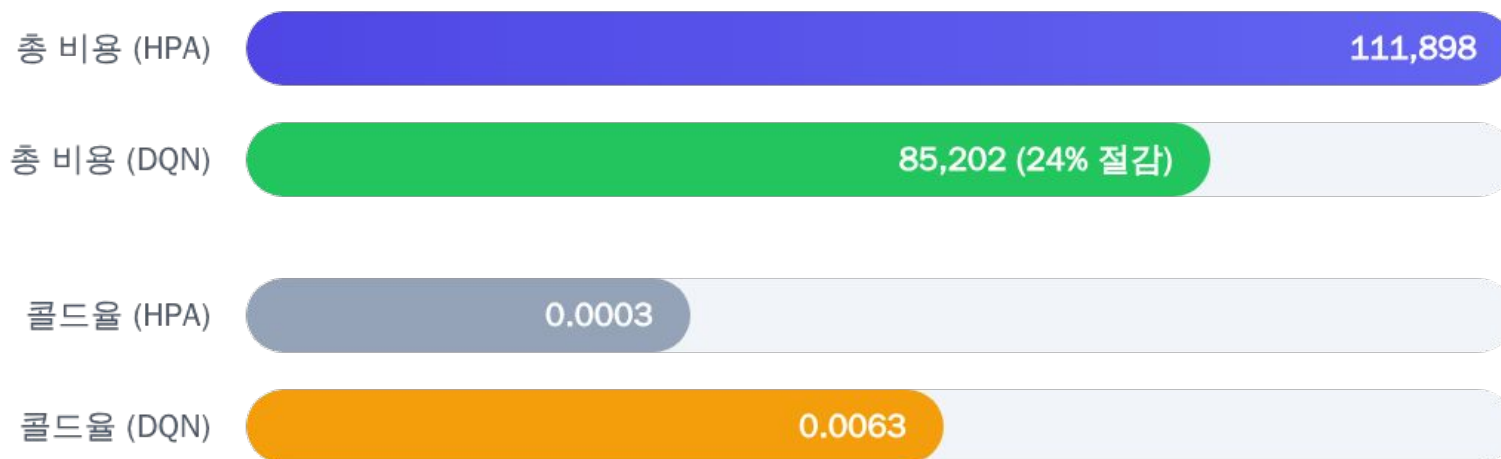
가중치가 높아질수록 콜드스타트 회피가 우선되며,
낮아질수록 비용 절감이 우선됩니다.

에피소드: 1 에피소드 = 1일 (1,440 스텝)

알고리즘 및 실험 설정

구분	방법 (Method)	특징 및 목적
RL 알고리즘	DQN (Double+Dueling) / PPO	PyTorch 직접 구현 (외부 RL 라이브러리 미사용)
Baseline 1	Fixed (8)	고정 인스턴스 유지 (성능의 하한선)
Baseline 2	HPA (규칙 기반) / Seasonal	이용률 임계값 기반 스케일링 (실무 핵심 비교 대상)
Upper Bound	Oracle	미래 부하 정보를 사용한 이론적 최적 성능
평가 방법	Welch's t-test (95% CI)	통계적 유의성 검증 및 Reward 가중치 스윙 분석

결과 1: Steady 시나리오 (예측 가능한 부하)



DQN의 선제적 스케일링 성공: 하루 주기가 뚜렷한 패턴을 파악하여 피크 전에 인스턴스를 확장했습니다.

결과적으로 콜드스타트율을 유사하게 유지하면서 HPA 대비 비용을 **24% 절감**했습니다.

| 결과 2: Spiky 시나리오 (돌발 부하)

0.97

DQN 콜드스타트율

State 설계의 구조적 한계

처음에는 RL 학습의 실패처럼 보였습니다. HPA(0.27)에 비해 DQN의 콜드스타트율이 0.97로 극히 저조했기 때문입니다. 하지만 원인은 알고리즘이 아니었습니다. 스파이크를 예측할 수 있는 단서 자체가 **State에 존재하지 않았기** 때문입니다. 예측 불가능한 정보는 그 어떤 우수한 RL 알고리즘도 학습할 수 없다는 것을 의미합니다.

결과 3: Bursty 시나리오 (상시 고부하)

방법론	콜드스타트율 (95% CI)	총 비용	평균 이용률
Oracle	0.0000	5,954	0.214
HPA	0.0363	6,196	0.207
DQN	0.2551 [0.159, 0.351]	4,608	0.235
PPO	0.7554 [0.316, 1.195]	12,726	0.022

결과 해석: 본 환경에서는 Oracle과 유사한 성능을 내는 HPA 및 Seasonal 같은 반응형 정책이 유효합니다.
반면 DQN은 비용을 최소화하는(4,608) 최적의 정책으로 수렴했습니다.

| 알고리즘 분석: DQN vs PPO

DQN이 강점을 보인 이유

세 가지 시나리오 모두에서 DQN이 PPO보다 **통계적으로 유의미하게 우수**했습니다. (Welch's t-test 결과 $CI < 0$)

- **Replay Buffer:** 희귀한 스파이크 패턴도 버퍼를 통해 반복 학습하여 안정적인 대처가 가능했습니다.
- 이산적인(Discrete) Q-learning 구조가 분 단위 스케일링 환경에 더 적합했습니다.

PPO의 붕괴 원인

PPO는 5번의 Seed 중 4번이나 '인스턴스=0'이라는 극단적인 정책으로 수렴하며 탈출하지 못했습니다.

- 이산 액션과 단기 고빈도 보상 환경에서 PPO의 **높은 분산(Variance) Gradient**가 독이 되었습니다.
- 초반의 즉각적인 비용 보상이 뒤늦은 콜드스타트 페널티보다 더 크게 작용했습니다.

| 핵심 발견

예측 가능성의 중요성

RL 성능은 알고리즘의 고도화보다 **부하의 예측 가능성**에 더 크게 좌우됩니다.

Spiky 시나리오처럼 예측 단서가 State에 존재하지 않는 경우, 어떤 알고리즘도 한계를 보였습니다.

향후 Spiky 개선을 위해 이동평균, 외생 트리거 등 State에 예측 단서를 추가하는 것이 핵심일 것 입니다.

| 결론 및 가설 검증

H1: RL이 같은 비용에서 HPA보다 콜드스타트를 줄일 수 있을까?

네, DQN은 Steady 워크로드에서 유사한 콜드스타트율을 유지하며 HPA 대비 비용을 24% 절감했습니다.

H2: RL이 하루 주기 트래픽 패턴을 학습해 선제적으로 스케일링할 수 있을까?

네, 시간(sin/cos) 특징을 바탕으로 일중 패턴을 학습하여, 단순 반응형이 아닌 선제적 스케일링을 달성했습니다.

H3: Reward 가중치로 비용-SLA 균형을 명시적으로 조절할 수 있을까?

네, 보상 함수의 가중치 스위치를 통해, 비용과 SLA(콜드스타트) 간의 균형점을 명시적으로 조절할 수 있음을 증명했습니다.

결론적으로 RL은 패턴이 존재하는 환경에서 최적의 효율을 발휘하지만,

State 설계의 질이 알고리즘 자체보다 성능에 결정적인 영향을 미친다는 것을 프로젝트를 통해 명확히 깨우칠 수 있었습니다.

감사합니다

강화학습 프로젝트 | 서강대학교 인공지능학과 | 김한슬 (A74029)

GitHub: <https://github.com/1three/sogang-rl-autoscaling>